PLAN DE TRABAJO

Fases 11 a 18

Invitación de boda Pablo & Marisol · 28 de diciembre de 2026

Cómo se conectan las secciones nuevas, qué estilos usan y qué conceptos de Next.js aparecen

Énfasis en carrusel, galería collage y RSVP con Oracle Autonomous Database

10 de noviembre de 2026

Contenido

1. El plan en una página
2. Cómo se conectan las secciones
3. El sistema de estilos aplicado a lo nuevo
4. Conceptos de Next.js que vas a aprender
5. Fase 11 — Bienvenida
6. Fase 12 — Cronograma (línea ondulada)
7. Fase 13 — Vestimenta con imágenes
8. Fase 14 — Carrusel de Historia (a fondo)
9. Fase 15 — Galería collage + lightbox (a fondo)
10. Fase 16 — Despedida y agradecimientos
11. Fase 17 — Footer
12. Fase 18 — RSVP con Oracle (a fondo)
13. Orden de ejecución y checklist
14. Glosario de términos nuevos
15. Riesgos y decisiones pendientes

1. El plan en una página

La página ya tiene nueve secciones terminadas. Faltan nueve más, y la buena noticia es que **ninguna obliga a rehacer lo que ya existe**: cada una es un archivo nuevo que se inserta en `app/page.tsx` y, si aporta datos nuevos, un arreglo nuevo en `data/wedding-data.ts`.

Fase	Sección	Qué implica	Tipo
10	Ajustes	Icono `+` con imagen, flecha de Hospedaje, Historia justificada	estático
11	Bienvenida	Mensaje de los novios entre el Hero y la Historia	estático
12	Cronograma	Línea de tiempo ondulada con 6 hitos; vertical en móvil	estático + SVG
13	Vestimenta	Dos cards con foto (traje azul / vestido blanco)	estático
14	Carrusel	Historia con fundido cruzado y descripción por foto	`cliente`
15	Galería	Collage de tamaños variables + vista ampliada	`cliente`
16	Despedida	Agradecimiento antes del pie de página	estático
17	Footer	Cierre oscuro con foto y texto arena	estático
18	RSVP	Formulario, server action y persistencia en Oracle	`cliente` + `servidor`

> **Regla del proyecto que no cambia:** los componentes son de servidor por defecto. Solo las Fases 14, 15 y el formulario de la 18 llevan `"use client"`, porque son las únicas que necesitan estado, eventos o APIs del navegador.

Decisiones ya tomadas

Dato	Valor
Vestimenta del novio	Traje **azul** → los invitados evitan azul
Vestimenta de la novia	Vestido **blanco** → los invitados evitan blanco
Ceremonia	5:00 PM
Fotos con familiares y amigos	6:30 PM
Recepción	8:00 PM
Baile de los novios	9:00 PM
Partir el pastel	9:30 PM
Baile general	10:00 PM
Base de datos	**Oracle Autonomous Database (Always Free)**, en la nube

2. Cómo se conectan las secciones

Toda la página es un solo archivo de ensamblaje. No hay rutas, no hay navegación entre páginas: hay una lista ordenada de componentes y un conjunto de anclas (`id`) para saltar entre ellos.

2.1 El orden final de la página

```
app/page.tsx
├── Header                  (fijo, fuera de <main>)
└── main
    ├── Hero                #inicio        foto + nombres + fecha
    ├── Bienvenida          #bienvenida    mensaje de los novios      ← Fase 11
    ├── NuestraHistoria     #historia      texto + carrusel           ← Fase 14
    ├── CuentaRegresiva     #countdown     sección café + contador
    ├── ElGranDia           #gran-dia      lugar, horarios, mapa
    ├── Cronograma          #cronograma    línea de tiempo            ← Fase 12
    ├── CodigoVestimenta    #vestimenta    2 cards con foto           ← Fase 13
    ├── Hospedaje           #hospedaje     hoteles o "muy pronto"
    ├── Galeria             #galeria       collage + lightbox         ← Fase 15
    ├── PreguntasFrecuentes #faq           acordeón
    ├── Rsvp                #rsvp          formulario + Oracle        ← Fase 18
    ├── Despedida           #despedida     agradecimiento             ← Fase 16
    └── Footer              (sin ancla)    cierre oscuro              ← Fase 17
```

2.2 Las tres capas de cada sección

Entender esto es lo que hace que el proyecto escale sin desorden. Cada sección tiene siempre las mismas tres capas:

Capa	Vive en	Contiene	Ejemplo
Datos	`data/wedding-data.ts`	Textos, rutas de imágenes, horas	`welcome.message`
Presentación	`components/sections/*.tsx`	JSX y clases de Tailwind	`Bienvenida.tsx`
Ensamblaje	`app/page.tsx`	Solo el orden	`<Bienvenida />`

Si un texto cambia, se toca la capa 1. Si un color cambia, la capa 2. Si cambia el orden de la página, la capa 3. Nunca se mezclan.

2.3 Las anclas y el Header

El Header navega con `href="#historia"` . Ese enlace busca un elemento con `id="historia"` en la misma página. Dos detalles que ya nos mordieron una vez:

- El **label** puede llevar acento ("Galería"), el **href** y el **id** no (`#galeria`).
- Cada sección lleva `scroll-mt-24` : sin eso, el header fijo tapa el título al saltar.

> **El Header se está quedando sin espacio.** Hoy tiene 7 enlaces más el botón de RSVP. Al agregar Cronograma serían 8. En `lg` (1024 px) eso se aprieta. La decisión se toma en la Fase 12: lo más probable es quitar "Hospedaje" del menú (sigue accesible bajando) y dejar 7 enlaces + RSVP.

3. El sistema de estilos aplicado a lo nuevo

3.1 El patrón de sección

Todas las secciones comparten la misma estructura. Memorizarla es más útil que memorizar clases sueltas:

```
<section id="bienvenida" className="scroll-mt-24 bg-cream py-24">
  <div className="mx-auto max-w-3xl px-6 text-center">

    <p className="font-sans text-[11px] uppercase tracking-[0.45em] text-terracotta">
      Bienvenidos
    </p>

    <h2 className="mt-6 font-serif text-4xl leading-[1.1] text-coffee md:text-5xl">
      Título
    </h2>

    <div className="mx-auto mt-8 h-px w-16 bg-sand/60" />

    <p className="mt-8 font-sans text-base leading-relaxed text-ink/80">
      Texto de introducción
    </p>

  </div>
</section>
```

Ese bloque se repite en Historia, Countdown, El Gran Día, Hospedaje, Galería y FAQ. Las secciones nuevas lo reutilizan, así que todas se sienten parte del mismo sitio sin tener que inventar nada.

3.2 Fondos: cómo se alternan

El ritmo visual de la página se construye alternando fondos. Así queda con las secciones nuevas:

Sección	Fondo	Texto
Hero	foto + degradado	blanco/crema
Bienvenida	`bg-cream`	`ink` / `coffee`
Nuestra Historia	`bg-cream`	`ink` / `coffee`
Cuenta regresiva	`bg-coffee`	`cream`
El Gran Día	`bg-cream`	`ink`
Cronograma	`bg-sand/15`	`ink`
Vestimenta	`bg-cream`	`ink`
Hospedaje	`bg-sand/15`	`ink`
Galería	`bg-cream`	`ink`
FAQ	`bg-coffee`	`cream`
RSVP	`bg-cream`	`ink`
Despedida	`bg-coffee`	`cream`
Footer	`bg-ink` + foto	`sand`

> **Nunca dos fondos oscuros seguidos.** FAQ ya es café y Despedida también: entre ellas van RSVP (crema) y Galería (crema). Si dos secciones oscuras quedan contiguas, la página pierde ritmo y el texto cansa.

3.3 Clases nuevas que aparecerán

Clase	Para qué	Fase
`text-justify` / `md:text-justify`	Alinear ambos bordes del párrafo	10
`aspect-[4/5]`, `aspect-square`	Proporción fija para fotos	13, 15
`col-span-2`, `row-span-2`	Que una foto ocupe dos columnas o filas	15
`auto-rows-[220px]`	Altura base de cada fila del grid	15
`[grid-auto-flow:dense]`	Rellena huecos del collage	15
`opacity-0` / `opacity-100`	Fundido del carrusel	14
`transition-opacity duration-1000`	Que el fundido dure un segundo	14
`backdrop:bg-ink/70`	Fondo del modal nativo	15
`group-hover:-translate-y-1`	Que una card se levante al pasar el mouse	13
`focus-visible:outline-2`	Foco visible por teclado	14, 15, 18

3.4 Responsive: mobile-first

Todo lo que se escribe sin prefijo aplica al móvil, y los prefijos agregan reglas hacia arriba. Las secciones nuevas siguen la misma lógica:

Elemento	Móvil	Desktop
Cronograma	Lista vertical con línea a la izquierda	Onda horizontal con 6 puntos
Vestimenta	1 card debajo de la otra	2 cards lado a lado
Carrusel	Alto 320 px	Alto 400 px
Collage	2 columnas, filas de 180 px	4 columnas, filas de 220 px

4. Conceptos de Next.js que vas a aprender

4.1 Server Components y Client Components

En el App Router, un componente es de servidor por defecto: se ejecuta en el servidor, puede leer la base de datos y **no envía JavaScript al navegador**. Es ideal para texto y fotos.

Un componente se vuelve de cliente cuando se marca con `"use client"` en la primera línea. Solo entonces puede usar:

- estado (`useState`) y efectos (`useEffect`),
- eventos del usuario (`onClick`, `onChange`, `onMouseEnter`),
- APIs del navegador (`window`, `document`, `matchMedia`).

> **La regla no es "usar cliente para todo".** Cada componente cliente suma JavaScript que el invitado descarga, posiblemente con datos móviles. Por eso solo el carrusel, el lightbox y el formulario serán cliente; el resto queda en servidor.

4.2 Hidratación, y por qué importa

El servidor manda HTML ya armado. Después, React lo "hidrata": le adjunta los eventos y lo vuelve interactivo. Si el HTML del servidor y el primer render del cliente no coinciden, React avisa con un error de hidratación.

Por eso el contador muestra `--` al principio: en el servidor no existe `Date.now()` del visitante. La misma lógica aplica al carrusel: la foto activa inicial siempre es la 0, y el `setInterval` arranca después, dentro de un `useEffect`.

4.3 Los hooks que vas a usar

Hook	Qué hace	Dónde se usa
`useState`	Guarda un valor que, al cambiar, redibuja el componente	Foto activa, modal abierto, pausa
`useEffect`	Ejecuta código después del render y devuelve una limpieza	Intervalos, listeners, `matchMedia`
`useRef`	Guarda una referencia a un elemento del DOM sin redibujar	El `<dialog>` del lightbox
`useCallback`	Memoriza una función para que no cambie en cada render	La función "siguiente" del carrusel
`useActionState`	Conecta un formulario a una server action y expone estado y pendiente	Formulario RSVP

4.4 Server Actions

Una *server action* es una función que se ejecuta **en el servidor** pero se llama desde el navegador como si fuera local. Es la forma moderna de procesar un formulario sin escribir un endpoint a mano.

```
// 1. Se declara en un archivo con "use server" arriba
"use server";
export async function guardarRsvp(prev, formData) { /* ... */ }

// 2. Se pasa al formulario
<form action={formAction}> ... </form>
```

Next se encarga del transporte, de la serialización y de la protección CSRF. Tú solo escribes la lógica.

4.5 Variables de entorno

Las credenciales de Oracle **nunca** van en el código ni en un componente. Van en `.env.local` (que no se sube a Git) y se leen con `process.env` solo en el servidor.

Prefijo	Visible en	Uso
`ORACLE_USER`	Solo servidor	Usuario de la base
`NEXT_PUBLIC_APP_URL`	Servidor y navegador	Datos que pueden ser públicos

> **Todo lo que empiece con `NEXT_PUBLIC_` se incrusta en el JavaScript del navegador.** Si le pones ese prefijo a una contraseña, cualquiera la lee con F12. La contraseña de Oracle va sin prefijo.

4.6 next/image con y sin fill

Modo	Cuándo	Requisito
`width` + `height`	Íconos y logos	Valores fijos; la proporción se conserva
`fill`	Fotos que ocupan un contenedor	Padre con `relative` y altura o proporción

Con `fill` siempre se acompaña `object-cover` (recorta sin deformar) y `sizes` (le dice al navegador cuánto espacio ocupa la imagen en cada breakpoint).

5. Fase 11 — Bienvenida

Objetivo: recibir al invitado justo después de la foto de portada, con un mensaje breve de los novios. No lleva fotos ni JavaScript.

Datos

```
// data/wedding-data.ts
export const welcome = {
    kicker: "Bienvenidos",
    title: "Nos encantaría compartir este día contigo",
    message: "Con mucha alegría queremos invitarte a celebrar el comienzo de nuestra vida juntos. ...",
} as const;
```

La firma no se guarda: se toma de `couple`. Así, si algún día cambia un nombre, cambia en un solo lugar.

Componente

```
import { couple, welcome } from "@/data/wedding-data";

export default function Bienvenida() {
    return (
        <section id="bienvenida" className="scroll-mt-24 bg-cream py-24">
            <div className="mx-auto max-w-3xl px-6 text-center">
                <p className="font-sans text-[11px] uppercase tracking-[0.45em] text-terracotta">
                    {welcome.kicker}
                </p>

                <h2 className="mt-6 font-serif text-3xl leading-[1.2] text-coffee md:text-4xl">
                    {welcome.title}
                </h2>

                <div className="mt-8 flex items-center justify-center gap-4" aria-hidden="true">
                    <span className="h-px w-12 bg-sand" />
                    <span className="h-1.5 w-1.5 rotate-45 bg-terracotta" />
                    <span className="h-px w-12 bg-sand" />
                </div>

                <p className="mx-auto mt-8 max-w-2xl font-sans text-base leading-relaxed text-ink/80 md:text-
lg">
                    {welcome.message}
                </p>

                <p className="mt-10 font-serif text-2xl italic text-terracotta">
                    {couple.groom} & {couple.bride}
                </p>
            </div>
        </section>
    );
}
```

Detalles que vale la pena entender

- `max-w-3xl`: un mensaje corto se lee mejor en 768 px que en 1152. Es la misma lógica que el FAQ.
- `text-center` en el contenedor: los párrafos heredan la alineación. Para un texto de 3–4 líneas se ve bien; para un texto largo no (por eso Historia usa justificado a la izquierda).

- El ornamento (línea, rombo, línea) es puro CSS: un `span` de 6 × 6 px con `rotate-45` se ve como un rombo. Lleva `aria-hidden` porque es decorativo.
- La firma usa `italic` y la serif: es el gesto editorial que cierra el bloque.

> **Efecto colateral que hay que corregir:** la flecha para bajar del Hero apunta a `#historia`. Al insertar Bienvenida en medio, debe apuntar a `#bienvenida`; si no, salta una sección de largo.

6. Fase 12 — Cronograma

Objetivo: mostrar los seis momentos del día sobre una línea ondulada, con la hora y una descripción corta.

Datos

```
export type ScheduleItem = {
    time: string;      // "5:00 PM"
    title: string;     // "Ceremonia"
    description: string;
};

export const schedule: ScheduleItem[] = [
    { time: "5:00 PM",  title: "Ceremonia",        description: "Nos damos el sí." },
    { time: "6:30 PM",  title: "Fotos",            description: "Con familiares y amigos." },
    { time: "8:00 PM",  title: "Recepción",        description: "Cena y brindis." },
    { time: "9:00 PM",  title: "Baile de los novios", description: "El primer baile." },
    { time: "9:30 PM",  title: "Pastel",           description: "El corte y el postre." },
    { time: "10:00 PM", title: "Baile general",    description: "A celebrar." },
];
```

Cómo se dibuja la onda

Dos piezas independientes que se alinean por porcentaje:

1. Un **SVG** con una sola ruta de curvas, estirado al ancho completo.
2. Los **puntos**, posicionados en absoluto con `left-[8%]`, `left-[25%]`, etc., alternando arriba y abajo.

```
<div className="relative hidden h-40 md:block">
  <svg viewBox="0 0 1000 120" preserveAspectRatio="none"
       className="absolute inset-0 h-full w-full text-sand">
    <path
      d="M0,60 C125,10 125,10 250,60 S375,110 500,60 S625,10 750,60 S875,110 1000,60"
      fill="none" stroke="currentColor" strokeWidth="1.5" />
  </svg>

  {/* Un punto por hito, posicionado por porcentaje */}
  <span className="absolute left-[8%] top-[30%] h-2.5 w-2.5 -translate-x-1/2 rounded-full bg-terracotta" />
  {/* ... */}
</div>
```

Qué significa cada atributo del SVG

Atributo	Qué hace
`viewBox="0 0 1000 120"`	Define un sistema de coordenadas interno de 1000 × 120
`preserveAspectRatio="none"`	Estira el dibujo al contenedor, sin mantener proporción
`d="M0,60 C..."`	La ruta: `M` mueve el lápiz, `C` dibuja una curva de Bézier, `S` continúa suavemente
`fill="none"`	Solo línea, sin relleno
`stroke="currentColor"`	Usa el color de texto del contenedor (aquí `text-sand`)

> **El móvil no usa la onda.** A 375 px de ancho, seis puntos en una onda quedan ilegibles. En móvil se muestra una línea vertical a la izquierda con los hitos apilados; la onda aparece desde `md`.

7. Fase 13 — Vestimenta con imágenes

Objetivo: dos cards, una con el traje y otra con el vestido, con una nota de color debajo de cada imagen.

Datos

```
export const dressCode = {
    title: "Formal",
    description: "Te pedimos venir con vestimenta formal.",
    groom: {
        label: "Ellos",
        image: "/images/vestimenta-traje.jpg",
        alt: "Traje azul del novio",
        note: "Evita el azul: lo llevará el novio.",
    },
    bride: {
        label: "Ellas",
        image: "/images/vestimenta-vestido.jpg",
        alt: "Vestido blanco de la novia",
        note: "Evita el blanco: lo llevará la novia.",
    },
} as const;
```

Patrón de la card

```
<figure className="overflow-hidden rounded-2xl border border-sand/60 bg-cream">
  <div className="group relative aspect-[4/5] overflow-hidden">
    <Image src={dressCode.groom.image} alt={dressCode.groom.alt}
           fill sizes="(min-width: 768px) 50vw, 100vw"
           className="object-cover transition-transform duration-500 group-hover:scale-105" />
  </div>
  <figcaption className="p-6">
    <p className="font-serif text-xl text-coffee">{dressCode.groom.label}</p>
    <p className="mt-1 font-sans text-sm text-ink/70">{dressCode.groom.note}</p>
  </figcaption>
</figure>
```

> **Por qué `figure` y `figcaption`:** una imagen con su descripción es exactamente lo que representan esas etiquetas. Un lector de pantalla anuncia la nota como parte de la imagen, no como un párrafo suelto.

Conteo de columnas: si los dos modelos tienen la misma proporción (4/5), las cards quedan parejas. Si una foto es horizontal, usa `object-cover` para recortarla al marco y que no rompa la simetría.

8. Fase 14 — Carrusel de Historia (a fondo)

Es el primer componente realmente interactivo del sitio. Vale la pena entenderlo bien porque el mismo patrón se repite en cualquier carrusel que hagas después.

8.1 La idea en una frase

Todas las fotos están montadas al mismo tiempo, una encima de otra; solo cambia la opacidad de la activa. No hay desplazamiento, no hay posición, no hay transformaciones: solo `opacity`. Eso es lo que produce el fundido.

```
Contenedor (alto fijo, overflow-hidden)
┌──────────────────────────────────────┐
│ figura 0   opacity-0   (invisible)   │
│ figura 1   opacity-100 (visible)     │  ← todas están, apiladas
│ figura 2   opacity-0   (invisible)   │
│ figura 3   opacity-0   (invisible)   │
└──────────────────────────────────────┘

        ▲ cambia el índice cada 6 s
        └── clase opacity-0 / opacity-100 + transition-opacity
```

8.2 Ventajas frente al carrusel deslizante

Aspecto	Fundido (elegido)	Deslizante
JS necesario	Un índice y un intervalo	Índice, offset, gestos táctiles
Salto de layout	Ninguno: el contenedor tiene alto fijo	Frecuente si no se calcula el ancho
Accesibilidad	Simple: se ocultan las inactivas	Hay que gestionar foco y scroll
Estética	Elegante, editorial	Más "app"

8.3 Datos

```
export type HistoryPhoto = {
    src: string;
    alt: string;
    caption: string;   // descripción que aparece bajo la foto
};

export const historyPhotos: HistoryPhoto[] = [
    { src: "/images/historia-01.jpg", alt: "Pablo y Marisol en el parque",
      caption: "Nuestra primera salida, 2020." },
    // ...
];
```

8.4 El componente completo, explicado

```
"use client";

import Image from "next/image";
import { useCallback, useEffect, useState } from "react";
import type { HistoryPhoto } from "@/data/wedding-data";

type Props = { photos: HistoryPhoto[]; intervalo?: number };

export default function HistoriaCarrusel({ photos, intervalo = 6000 }: Props) {
    const [activa, setActiva] = useState(0);
    const [pausado, setPausado] = useState(false);
    const [reducir, setReducir] = useState(false);

    /* Avanza circularmente: 0,1,2,3,0,1... */
    const siguiente = useCallback(() => {
        setActiva((i) => (i + 1) % photos.length);
    }, [photos.length]);

    /* Respeta la preferencia del sistema: sin animaciones automáticas */
    useEffect(() => {
        const mq = window.matchMedia("(prefers-reduced-motion: reduce)");
        setReducir(mq.matches);
        const onChange = (e: MediaQueryListEvent) => setReducir(e.matches);
        mq.addEventListener("change", onChange);
        return () => mq.removeEventListener("change", onChange);
    }, []);

    /* El reloj: se reinicia si se pausa y se limpia al desmontar */
    useEffect(() => {
        if (pausado || reducir || photos.length <= 1) return;
        const id = setInterval(siguiente, intervalo);
        return () => clearInterval(id);
    }, [pausado, reducir, intervalo, siguiente, photos.length]);

    return (
        <div className="relative"
             onMouseEnter={() => setPausado(true)}
             onMouseLeave={() => setPausado(false)}
             onFocus={() => setPausado(true)}
             onBlur={() => setPausado(false)}>

            <div className="relative h-80 w-full overflow-hidden rounded-2xl md:h-[400px]">
                {photos.map((photo, i) => (
                    <figure key={photo.src}
                            aria-hidden={i !== activa}
                            className={`absolute inset-0 transition-opacity duration-1000 ${
                                i === activa ? "opacity-100" : "opacity-0"
                            }`}>
                        <Image src={photo.src} alt={photo.alt} fill
                               sizes="(min-width: 768px) 50vw, 100vw"
                               className="object-cover" />

                        <div className="absolute inset-0 bg-linear-to-t from-ink/75 via-transparent to-
transparent" />

                        <figcaption className="absolute inset-x-0 bottom-0 p-5 font-sans text-sm text-cream">
                            {photo.caption}
                        </figcaption>
                    </figure>
                ))}
```

```
            </div>

            <div className="mt-5 flex justify-center gap-2">
                {photos.map((photo, i) => (
                    <button key={photo.src} type="button"
                            onClick={() => setActiva(i)}
                            aria-label={`Ver foto ${i + 1} de ${photos.length}`}
                            aria-current={i === activa ? "true" : undefined}
                            className={`h-2 rounded-full transition-all duration-300 ${
                                i === activa ? "w-6 bg-terracotta" : "w-2 bg-sand"
                            }`} />
                ))}
            </div>
        </div>
    );
}
```

8.5 Por qué cada decisión

Detalle	Razón
`absolute inset-0` en cada figura	Apila todas en el mismo lugar; sin esto se verían una debajo de otra
Alto en el contenedor (`h-80 md:h-[400px]`)	Con `fill` la imagen necesita un padre con altura; además evita el salto de layout
`aria-hidden` en las inactivas	Un lector de pantalla no debe leer cuatro descripciones a la vez
`useCallback` en `siguiente`	Si la función cambiara en cada render, el `useEffect` reiniciaría el intervalo constantemente
`return () => clearInterval(id)`	Sin la limpieza, al desmontar quedaría un reloj corriendo (fuga de memoria)
`pausado` y `reducir`	Nadie debe perder una foto porque el mouse pasó por encima, ni sufrir animaciones si pidió reducirlas
`(i + 1) % length`	El módulo hace circular el índice: después del último vuelve al primero
Puntos como `<button>`	Son navegables con teclado; un `div` con `onClick` no lo es

8.6 Errores típicos

Síntoma	Causa
Se ven todas las fotos amontonadas	Falta `position: absolute` en cada figura
El carrusel salta en cada cambio	El contenedor no tiene alto fijo
El intervalo se acelera solo	Falta `useCallback` o la dependencia en el `useEffect`
La descripción no se desvanece	El `figcaption` quedó fuera de la figura que cambia de opacidad
Nunca avanza	`pausado` quedó en `true`, o `photos.length` es 1

9. Fase 15 — Galería collage + lightbox (a fondo)

9.1 El modelo de datos

```
export type GalleryPhoto = {
    src: string;
    alt: string;
    caption?: string;
    size?: "normal" | "ancho" | "alto" | "grande";
};
```

El tamaño no se adivina con CSS: se decide en los datos. El componente traduce esa palabra a clases.

```
const clasesDeTamano = {
    normal: "",
    ancho:  "col-span-2",
    alto:   "row-span-2",
    grande: "col-span-2 row-span-2",
} as const;
```

9.2 El grid que lo hace posible

```
<div className="grid grid-cols-2 auto-rows-[180px] gap-3
                md:grid-cols-4 md:auto-rows-[220px] md:gap-4
                [grid-auto-flow:dense]">

  {photos.map((photo) => (
    <button key={photo.src}
           className={`group relative overflow-hidden rounded-xl ${clasesDeTamano[photo.size ??
"normal"]}`}>
      <Image src={photo.src} alt={photo.alt} fill
             sizes="(min-width: 768px) 25vw, 50vw"
             className="object-cover transition-transform duration-500 group-hover:scale-105" />
    </button>
  ))}
</div>
```

Pieza	Qué hace
`auto-rows-[180px]`	Cada fila mide 180 px. Con `row-span-2` una foto mide 360 + separación
`col-span-2`	La foto ocupa dos columnas: se ve horizontal
`[grid-auto-flow:dense]`	Rellena huecos con las fotos siguientes en vez de dejar espacios vacíos
`aspect-*` desaparece	En un collage la proporción la define el span, no un aspect ratio fijo

> **Sin** `dense` el grid respeta el orden y deja agujeros cuando una foto grande no cabe. Con `dense` reacomoda las siguientes para llenar el hueco. Es lo que hace que un collage se vea intencional.

9.3 El lightbox con <dialog>

HTML ya tiene un elemento para esto: `<dialog>`. Con `showModal()` se abre encima de todo, bloquea el resto de la página y captura `Escape` automáticamente. No hay que reinventar el foco ni el fondo.

```
"use client";

import Image from "next/image";
import { useEffect, useRef } from "react";
import type { GalleryPhoto } from "@/data/wedding-data";

type Props = {
    photos: GalleryPhoto[];
    index: number | null;               // null = cerrado
    onClose: () => void;
    onNavigate: (i: number) => void;
};

export default function Lightbox({ photos, index, onClose, onNavigate }: Props) {
    const ref = useRef<HTMLDialogElement>(null);

    /* Abre y cierra el diálogo cuando cambia el índice */
    useEffect(() => {
        const dialog = ref.current;
        if (!dialog) return;

        if (index !== null && !dialog.open) {
            dialog.showModal();
            document.body.style.overflow = "hidden";
        }
        if (index === null && dialog.open) {
            dialog.close();
            document.body.style.overflow = "";
        }
    }, [index]);

    /* Flechas del teclado mientras está abierto */
    useEffect(() => {
        if (index === null) return;
        const onKey = (e: KeyboardEvent) => {
            if (e.key === "ArrowRight") onNavigate((index + 1) % photos.length);
            if (e.key === "ArrowLeft")  onNavigate((index - 1 + photos.length) % photos.length);
        };
        window.addEventListener("keydown", onKey);
        return () => window.removeEventListener("keydown", onKey);
    }, [index, photos.length, onNavigate]);

    /* Escape y clic en el fondo cierran */
    return (
        <dialog
            ref={ref}
            onClose={onClose}
            onClick={(e) => { if (e.target === e.currentTarget) onClose(); }}
            className="m-auto w-full max-w-5xl rounded-2xl bg-ink/95 p-0
                       backdrop:bg-ink/80 backdrop:backdrop-blur-sm">

            {index !== null && (
                <figure className="relative">
                    <div className="relative aspect-[3/2] w-full">
                        <Image src={photos[index].src} alt={photos[index].alt} fill
                               sizes="90vw" className="object-contain" />
                    </div>

                    <figcaption className="flex items-center justify-between gap-4 p-4 font-sans text-sm
text-cream">
                        <span>{photos[index].caption ?? photos[index].alt}</span>
                        <button type="button" onClick={onClose}
```

```
                                    className="rounded-full border border-cream/30 px-4 py-1
                                               text-xs uppercase tracking-[0.2em]">
                                Cerrar
                            </button>
                        </figcaption>
                    </figure>
                )}
            </dialog>
        );
}
```

9.4 Qué resuelve el diálogo nativo

Problema	Cómo lo resuelve
Cerrar con `Escape`	Nativo, sin listener
El resto de la página no se puede clickear	La capa modal lo bloquea
El foco se va detrás del modal	`showModal()` lo encierra
Fondo oscuro	Pseudo-elemento `::backdrop`, expuesto en Tailwind como `backdrop:`

Detalle importante: el diálogo se monta siempre, aunque esté cerrado. No se usa `{index !== null && <dialog>...}`, porque el elemento debe existir para poder llamar `showModal()` desde el efecto. Lo que cambia es su contenido.

9.5 Accesibilidad de la galería

- Cada foto es un `<button>`, no un `div`: se abre con teclado.
- El botón lleva `aria-label` describiendo la foto ("Ampliar: Pablo y Marisol en el jardín").
- Al cerrar, el foco vuelve solo al botón que abrió (el navegador lo hace con `dialog.close()`).
- Se bloquea el scroll del fondo mientras el modal está abierto.
- Las flechas izquierda/derecha funcionan con el teclado.

10. Fase 16 — Despedida y agradecimientos

Mismo patrón que Bienvenida, en fondo café para cerrar con fuerza antes del footer.

```
<section id="despedida" className="scroll-mt-24 bg-coffee py-24">
  <div className="mx-auto max-w-3xl px-6 text-center">
    <p className="font-sans text-[11px] uppercase tracking-[0.45em] text-terracotta">
      Gracias
    </p>
    <h2 className="mt-6 font-serif text-3xl leading-[1.2] text-cream md:text-4xl">
      {farewell.title}
    </h2>
    <div className="mx-auto mt-8 h-px w-16 bg-terracotta/60" />
    <p className="mt-8 font-sans text-base leading-relaxed text-cream/75">
      {farewell.message}
    </p>
  </div>
</section>
```

Detalle de contraste: sobre café, el texto va en `text-cream/75`, no en `text-cream` puro. Un párrafo largo en blanco total sobre fondo oscuro cansa la vista; bajarlo al 75 % mantiene la legibilidad y suaviza.

11. Fase 17 — Footer

Cierre de página con foto de fondo, degradado y tres bloques: monograma, fecha y un mensaje corto.

```
<footer className="relative overflow-hidden">
  <Image src={featuredPhotos.hero} alt="" fill sizes="100vw"
         className="object-cover" />

  <div className="absolute inset-0 bg-ink/85" />

  <div className="relative z-10 mx-auto max-w-6xl px-6 py-16 text-center">
    <p className="font-serif text-3xl text-cream">
      {couple.groom} & {couple.bride}
    </p>
    <p className="mt-3 font-sans text-[11px] uppercase tracking-[0.3em] text-sand">
      {event.dateLabel}
    </p>
    <p className="mt-6 font-sans text-xs text-sand/70">
      Hecho con cariño para celebrar con ustedes.
    </p>
  </div>
</footer>
```

Decisión de rendimiento: el footer reutiliza la foto del Hero en lugar de cargar una nueva. A esa escala y con un velo del 85 % encima, la diferencia es invisible, y el navegador ya la tiene en caché. En Next, además, la variante servida es la misma si el `sizes` coincide.

12. Fase 18 — RSVP con Oracle (a fondo)

12.1 Qué es Autonomous Database "Always Free"

Oracle Autonomous Database es un servicio administrado: Oracle se encarga de parches, respaldos y ajuste. La capa **Always Free** es gratuita sin fecha de expiración e incluye 20 GB de almacenamiento y dos instancias. Se administra desde el navegador (Database Actions), así que no necesitas instalar nada para empezar.

> **Por qué encaja con este proyecto:** el sitio se publica en Vercel, que no puede alojar un servidor Oracle. Una base en la nube resuelve eso y te deja practicar SQL real desde el primer día.

12.2 Arquitectura del flujo

```
Invitado
   │  llena el formulario y envía
   ▼
Rsvp.tsx  ("use client")            ← useActionState, estados de error y "enviando"
   │
   │  form action
   ▼
actions/rsvp.ts  ("use server")     ← valida, arma los binds, ejecuta
   │
   │  pool.execute(sql, binds)
   ▼
lib/oracle.ts                       ← un solo pool para toda la app
   │
   │  TCPS 1522 (cifrado)
   ▼
Oracle Autonomous Database (Always Free)
   └── tabla RSVP
```

12.3 Preparación de la base (una sola vez)

1. Crear la instancia en Oracle Cloud con la región más cercana (Querétaro, si está disponible en tu cuenta).
2. Guardar la contraseña del usuario `ADMIN` en un lugar seguro.
3. Entrar a **Database Actions** → **SQL** y ejecutar el DDL de abajo.
4. En **Database connection**, copiar la cadena de conexión TLS (protocolo `tcps`, puerto 1522).

```
CREATE TABLE RSVP (
    ID             NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    NOMBRE         VARCHAR2(120)  NOT NULL,
    CORREO         VARCHAR2(160),
    TELEFONO       VARCHAR2(40),
    ASISTE         CHAR(1)        DEFAULT 'S' NOT NULL,
    ACOMPANANTES   NUMBER(2)      DEFAULT 0  NOT NULL,
    RESTRICCIONES  VARCHAR2(400),
    MENSAJE        VARCHAR2(600),
    CREADO_EN      TIMESTAMP      DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT RSVP_ASISTE_CK CHECK (ASISTE IN ('S','N')),
    CONSTRAINT RSVP_ACOMP_CK  CHECK (ACOMPANANTES BETWEEN 0 AND 10)
);

-- Consulta de control
SELECT COUNT(*) AS total,
       SUM(CASE WHEN ASISTE = 'S' THEN 1 + ACOMPANANTES ELSE 0 END) AS personas
FROM RSVP;
```

12.4 Instalar el driver

```
npm install oracledb
```

El driver moderno trabaja en **modo thin**: JavaScript puro, sin necesidad de instalar el Oracle Client. Eso es lo que permite usarlo en Vercel sin compilar nada nativo.

12.5 Variables de entorno

```
# .env.local  (nunca se sube a Git)
ORACLE_USER=ADMIN
ORACLE_PASSWORD=tu_password
ORACLE_CONNECT_STRING=(description=(retry_count=20)(retry_delay=3)
  (address=(protocol=tcps)(port=1522)(host=adb.mx-queretaro-1.oraclecloud.com))
  (connect_data=(service_name=g12345_miboda_high.adb.oraclecloud.com))
  (security=(ssl_server_dn_match=yes)))
```

La cadena se puede escribir en una sola línea; aquí está partida por legibilidad. Es la que copia la consola de Oracle, no hay que inventarla.

12.6 El pool de conexiones

```
// lib/oracle.ts
import oracledb from "oracledb";

declare global {
    var __oraclePool: oracledb.Pool | undefined;
}

export async function getPool() {
    if (!globalThis.__oraclePool) {
        globalThis.__oraclePool = await oracledb.createPool({
            user: process.env.ORACLE_USER,
            password: process.env.ORACLE_PASSWORD,
            connectString: process.env.ORACLE_CONNECT_STRING,
            poolMin: 1,
            poolMax: 4,
            poolTimeout: 60,
        });
    }
    return globalThis.__oraclePool;
}
```

Detalle	Por qué
Pool y no una conexión por request	Abrir una conexión cuesta cientos de milisegundos; el pool las reutiliza
`globalThis`	En desarrollo, Next recarga el módulo en cada cambio. Sin esto, cada recarga crearía un pool nuevo hasta agotar el límite de sesiones de Oracle
`poolMax: 4`	Una boda no necesita más. El plan gratuito tiene un límite de sesiones y conviene no acercarse
`import oracledb`	El driver solo se ejecuta en el servidor. Nunca debe importarse desde un componente cliente

12.7 La server action

```
// app/actions/rsvp.ts
"use server";

import { getPool } from "@/lib/oracle";

export type RsvpState = { ok: boolean; message: string };

export async function guardarRsvp(
    _prev: RsvpState,
    formData: FormData,
): Promise<RsvpState> {
    const nombre = String(formData.get("nombre") ?? "").trim();
    const correo = String(formData.get("correo") ?? "").trim();
    const telefono = String(formData.get("telefono") ?? "").trim();
    const asiste = formData.get("asiste") === "no" ? "N" : "S";
    const acompanantes = Number(formData.get("acompanantes") ?? 0);
    const restricciones = String(formData.get("restricciones") ?? "").trim();
    const mensaje = String(formData.get("mensaje") ?? "").trim();

    if (nombre.length < 3) {
        return { ok: false, message: "Escribe tu nombre completo." };
    }
    if (!Number.isInteger(acompanantes) || acompanantes < 0 || acompanantes > 10) {
        return { ok: false, message: "El número de acompañantes no es válido." };
    }

    try {
        const pool = await getPool();
        await pool.execute(
            `INSERT INTO RSVP
                 (NOMBRE, CORREO, TELEFONO, ASISTE, ACOMPANANTES, RESTRICCIONES, MENSAJE)
             VALUES (:nombre, :correo, :telefono, :asiste, :acompanantes, :restricciones, :mensaje)`,
            { nombre, correo, telefono, asiste, acompanantes, restricciones, mensaje },
            { autoCommit: true },
        );
        return { ok: true, message: "¡Gracias! Tu confirmación quedó registrada." };
    } catch (error) {
        console.error("Error al guardar RSVP", error);
        return { ok: false, message: "No pudimos guardar tu confirmación. Intenta de nuevo." };
    }
}
```

12.8 Bind variables: la parte de seguridad

Nunca se concatena texto del usuario dentro del SQL. Se usan **bind variables**: el SQL lleva `:nombre` y los valores viajan aparte.

```
// ❌ Mal: inyección SQL. Con nombre = "x'); DROP TABLE RSVP; --" se borra la tabla
await pool.execute(`INSERT INTO RSVP (NOMBRE) VALUES ('${nombre}')`);

// ✅ Bien: el valor nunca se interpreta como SQL
await pool.execute("INSERT INTO RSVP (NOMBRE) VALUES (:nombre)", { nombre });
```

Además de la seguridad, los binds permiten que Oracle reutilice el plan de ejecución: es más rápido.

12.9 El formulario en el cliente

```
"use client";

import { useActionState } from "react";
import { guardarRsvp, type RsvpState } from "@/app/actions/rsvp";

const inicial: RsvpState = { ok: false, message: "" };

export default function RsvpForm() {
    const [estado, formAction, pendiente] = useActionState(guardarRsvp, inicial);

    return (
        <form action={formAction} className="mx-auto max-w-xl space-y-5">
            <div>
                <label htmlFor="nombre" className="font-sans text-xs uppercase tracking-[0.2em] text-ink/70">
                    Nombre completo
                </label>
                <input id="nombre" name="nombre" required minLength={3}
                       className="mt-2 w-full rounded-xl border border-sand/70 bg-cream px-4 py-3
                                  font-sans text-base text-ink outline-none
                                  focus:border-terracotta" />
            </div>

            <fieldset>
                <legend className="font-sans text-xs uppercase tracking-[0.2em] text-ink/70">
                    ¿Podrás acompañarnos?
                </legend>
                <div className="mt-2 flex gap-6 font-sans text-sm text-ink">
                    <label className="flex items-center gap-2">
                        <input type="radio" name="asiste" value="si" defaultChecked /> Sí
                    </label>
                    <label className="flex items-center gap-2">
                        <input type="radio" name="asiste" value="no" /> No
                    </label>
                </div>
            </fieldset>

            <div>
                <label htmlFor="acompanantes" className="font-sans text-xs uppercase tracking-[0.2em] text-
ink/70">
                    Acompañantes
                </label>
                <input id="acompanantes" name="acompanantes" type="number" min={0} max={10} defaultValue={0}
                       className="mt-2 w-full rounded-xl border border-sand/70 bg-cream px-4 py-3
                                  font-sans text-base text-ink outline-none focus:border-terracotta" />
            </div>

            <div>
                <label htmlFor="restricciones" className="font-sans text-xs uppercase tracking-[0.2em] text-
ink/70">
                    Restricciones alimenticias
                </label>
                <textarea id="restricciones" name="restricciones" rows={3}
                          className="mt-2 w-full rounded-xl border border-sand/70 bg-cream px-4 py-3
                                     font-sans text-base text-ink outline-none focus:border-terracotta" />
            </div>

            <button type="submit" disabled={pendiente}
                    className="w-full rounded-full bg-terracotta px-6 py-3 font-sans text-xs
                               uppercase tracking-[0.2em] text-white transition-colors duration-300
                               hover:bg-cinnamon disabled:opacity-60">
```

```
                {pendiente ? "Enviando..." : "Confirmar asistencia"}
            </button>

            {estado.message && (
                <p role="status"
                   className={`font-sans text-sm ${estado.ok ? "text-cinnamon" : "text-red-700"}`}>
                    {estado.message}
                </p>
            )}
        </form>
    );
}
```

Detalle	Por qué
`useActionState`	Devuelve el resultado de la acción, la función para el `action` y el estado pendiente
`action={formAction}`	Conecta el formulario a la acción del servidor sin `onSubmit` ni `fetch`
`disabled={pendiente}`	Evita dobles envíos por doble clic
`required`, `minLength`, `type="number"`	La primera validación es del navegador; la del servidor es la que cuenta
`role="status"`	Los lectores de pantalla anuncian el resultado sin mover el foco
`name` en cada campo	Es la llave con la que `formData.get()` encuentra el valor

12.10 Probar antes de desplegar

Un script de 15 líneas confirma que la conexión y la tabla funcionan, sin pasar por el navegador:

```
// scripts/probar-oracle.mjs
import oracledb from "oracledb";

const conn = await oracledb.getConnection({
    user: process.env.ORACLE_USER,
    password: process.env.ORACLE_PASSWORD,
    connectString: process.env.ORACLE_CONNECT_STRING,
});

const r = await conn.execute("SELECT SYSTIMESTAMP FROM dual");
console.log("Conectado:", r.rows[0][0]);
await conn.close();
```

```
node --env-file=.env.local scripts/probar-oracle.mjs
```

12.11 Despliegue en Vercel

1. Project Settings → Environment Variables: cargar `ORACLE_USER`, `ORACLE_PASSWORD` y `ORACLE_CONNECT_STRING`.
2. Nunca marcarlas como `NEXT_PUBLIC_`.
3. Habilitar el acceso desde cualquier IP en la lista de control de acceso de la base, o restringirlo si el proveedor ofrece IPs fijas.
4. Después del primer despliegue, enviar una confirmación de prueba y verificar con `SELECT`.

12.12 Privacidad y abuso

- Pide solo lo necesario: nombre, asistencia, acompañantes y restricciones. El correo y el teléfono son opcionales.
- No registres datos personales en `console.log`.
- Un formulario público recibe spam. Cuando funcione, se puede agregar un campo trampa oculto y un límite por IP.
- El dueño de los datos debe poder borrarlos: un `DELETE` por nombre es suficiente para una boda.

13. Orden de ejecución y checklist

Fase	Entregable	Cómo se verifica
10	Iconos y justificado	El `+` gira a ×; la flecha se desliza; el texto justifica solo en desktop
11	Bienvenida	Aparece tras el Hero; la flecha del Hero baja a la sección correcta
12	Cronograma	6 hitos con las horas correctas; vertical en móvil
13	Vestimenta	2 cards parejas; nota de color visible
14	Carrusel	Cambia cada 6 s, se pausa al pasar el mouse, los puntos funcionan con Tab + Enter
15	Collage + lightbox	Tamaños variados sin huecos; modal abre, navega con flechas y cierra con Escape
16	Despedida	Texto legible sobre café; sin fondo oscuro contiguo
17	Footer	Foto con velo; texto arena legible
18	RSVP + Oracle	Envío real, mensaje de éxito, fila visible con `SELECT`

Comprobaciones antes de publicar

- `npx tsc --noEmit` sin errores.
- `npx eslint .` sin errores ni avisos.
- Probar en un teléfono real: menú, carrusel, collage, formulario.
- Revisar el peso de las imágenes (Hero < 500 KB, galería < 300 KB cada una).
- Confirmar que ninguna credencial aparece en el JavaScript del navegador.

14. Glosario de términos nuevos

Término	Significado
Fundido cruzado	Una imagen desaparece mientras otra aparece en el mismo lugar, cambiando la opacidad
Lightbox	Capa que muestra una imagen ampliada sobre el resto de la página
Collage	Cuadrícula donde las fotos ocupan distinto número de columnas o filas
Server Action	Función que corre en el servidor y se invoca desde un formulario o un botón
Bind variable	Marcador `:nombre` en el SQL; el valor viaja aparte y no se interpreta como código
Pool	Conjunto de conexiones abiertas que se reutilizan en vez de crear una por consulta
DDL / DML	DDL crea estructura (`CREATE TABLE`); DML manipula datos (`INSERT`, `SELECT`)
TNS / TCPS	El "nombre" y la dirección cifrada de una base Oracle
Always Free	Capa gratuita sin fecha de expiración de Oracle Cloud
Modal	Ventana que bloquea la interacción con el resto de la página hasta cerrarse
Focus trap	Mantener el foco del teclado dentro del modal mientras está abierto
prefers-reduced-motion	Preferencia del sistema para reducir animaciones

15. Riesgos y decisiones pendientes

Riesgo	Impacto	Mitigación
No tener los textos de Bienvenida y Despedida	Se prototipa con texto de ejemplo	Reemplazar en `wedding-data.ts`; el componente no cambia
Fotos del traje y vestido sin llegar	Fase 13 bloqueada	Usar `PhotoPlaceholder` con la proporción 4/5
El Header se satura con 8 enlaces	Se aprieta en 1024 px	Quitar "Hospedaje" del menú o acortar etiquetas
Latencia de Oracle desde México	1–2 s por envío	Elegir región cercana; el formulario muestra "Enviando..."
Credenciales filtradas	Acceso total a la base	Variables sin `NEXT_PUBLIC_`, `.env.local` fuera de Git, rotar la contraseña si se expone
Spam en el formulario	Filas basura	Campo trampa, límite por IP y revisión manual
El carrusel distrae	Peor lectura en la Historia	Intervalo de 6 s, pausa al interactuar y respeto a `prefers-reduced-motion`

> **Regla para no perderte:** una fase, un archivo, una verificación en el navegador. Si algo falla, el problema está en lo último que pegaste, no en todo el proyecto.